

嵌入式面试题集

FreeRTOS • STM32

FreeRTOS 题目	25 题 (含答案)
STM32 题目	17 题 (含答案)
参考资料	正点原子《FreeRTOS开发指南V1.12》
适用岗位	嵌入式 Linux / IoT / 单片机 面试

2026-06-14

Part 1 FreeRTOS 面试题 (25题)

一、基础概念

Q01. 什么是 FreeRTOS? 它有哪些核心特点?

FreeRTOS 是一款免费、开源的嵌入式实时操作系统 (RTOS)，采用 MIT 许可证，可用于商业产品无需开源代码。

核心特点：可裁剪（通过 FreeRTOSConfig.h 按需启用）；可移植（支持 ARM Cortex-M/A、RISC-V 等架构）；任务数量和优先级数量不限；支持抢占式调度和时间片调度；提供队列、信号量、事件标志组、任务通知等同步机制；提供 5 种内存管理方案（heap_1~heap_5）。

★ FreeRTOS 使用 MIT 许可证，商业可用，不需要开源应用代码

Q02. FreeRTOS 的 FreeRTOSConfig.h 有什么作用? 列举 5 个常用宏。

FreeRTOSConfig.h 是 FreeRTOS 的系统配置文件，通过宏定义裁剪和配置功能。

```
configUSE_PREEMPTION = 1 // 启用抢占式调度
configCPU_CLOCK_HZ = 168000000 // CPU 主频 (STM32F407)
configTICK_RATE_HZ = 1000 // 系统节拍 1000Hz = 1ms/tick
configMAX_PRIORITIES = 32 // 最大任务优先级数
configTOTAL_HEAP_SIZE = (75*1024) // FreeRTOS 堆大小 75KB
configMINIMAL_STACK_SIZE = 128 // 最小栈 (字, 即 512 字节)
configMAX_SYSCALL_INTERRUPT_PRIORITY // 可调用API的最高中断优先级
```

二、任务管理

Q03. FreeRTOS 任务有哪四种状态? 各状态如何转换?

运行态 (Running)：正在占用 CPU 执行。

就绪态 (Ready)：已准备好，等待调度器分配 CPU。

阻塞态 (Blocked)：等待某事件（延时/信号量/队列），不占用 CPU。

挂起态 (Suspended)：被 vTaskSuspend() 挂起，不参与调度，需 vTaskResume() 唤醒。

转换：运行→阻塞（调用 vTaskDelay 等）；阻塞→就绪（事件发生）；运行→就绪（被更高优先级任务抢占）；运行/就绪→挂起（vTaskSuspend）；挂起→就绪（vTaskResume）。

★ 四态转换是调度器行为的基础，面试必考

Q04. xTaskCreate() 各参数的含义是什么?

函数原型：

```
BaseType_t xTaskCreate(  
    TaskFunction_t pvTaskCode, // 任务函数指针  
    const char* pcName, // 任务名称 (调试用)  
    uint16_t usStackDepth, // 栈大小 (单位: 字 = 4字节)  
    void* pvParameters, // 传给任务函数的参数  
    UBaseType_t uxPriority, // 优先级 (0最低)  
    TaskHandle_t* pxCreatedTask // 输出: 任务句柄 (可为NULL)  
);
```

返回 pdPASS 表示成功; errCOULD_NOT_ALLOCATE_REQUIRED_MEMORY 表示内存不足。

★ 栈大小单位是字 (word = 4字节), 不是字节! STM32 常用 128~512

Q05. 动态创建 (xTaskCreate) 与静态创建 (xTaskCreateStatic) 的区别?

动态创建: 栈和 TCB 由 FreeRTOS 从堆动态分配, 使用方便, 依赖堆配置。

静态创建: 栈和 TCB 由用户提供静态数组, 不依赖堆, 适合对内存使用严格的场合。需在 FreeRTOSConfig.h 定义 configSUPPORT_STATIC_ALLOCATION = 1, 并实现 vApplicationGetIdleTaskMemory() 钩子函数提供空闲任务静态内存。

Q06. FreeRTOS 任务优先级数值越大越高还是越低? 空闲任务优先级是多少?

数值越大优先级越高 (与 μ C/OS 相反), 范围 0 ~ (configMAX_PRIORITIES - 1)。

空闲任务 (Idle Task) 优先级为 0 (最低), 由调度器自动创建。用户任务不应长时间独占优先级 0, 否则动态内存释放会延迟。

Q07. vTaskDelete() 删除任务后, 内存是如何释放的?

若任务由 xTaskCreate() (动态) 创建, 任务的栈内存和 TCB 不会立即释放, 任务被标记为删除后进入等待删除状态, 由空闲任务 (prvIdleTask) 最终释放内存。因此系统必须保证空闲任务能定期运行。

若由 xTaskCreateStatic() (静态) 创建, TCB 和栈均为用户提供的静态数组, FreeRTOS 不释放, 由用户管理。

★ 删除任务后内存不是立即释放的, 需要空闲任务介入, 面试陷阱题

Q08. vTaskSuspend() 和 vTaskDelay() 有什么区别?

vTaskSuspend(handle): 挂起指定任务, 不参与调度, 需 vTaskResume() 恢复, 无超时。

vTaskDelay(ticks): 当前任务进入阻塞态, 等待指定 tick 数后自动回到就绪态, 无需外部唤醒。

使用场景: vTaskDelay 用于周期性延时; vTaskSuspend 用于条件控制暂停某任务。

Q09. 什么是任务控制块 (TCB) ? 包含哪些关键信息?

TCB (Task Control Block) 是 FreeRTOS 管理每个任务的数据结构 (TCB_t 类型)。

主要字段: pxTopOfStack (栈顶指针, 任务切换时保存/恢复 SP); xStateListItem (状态链表项, 用于插入就绪/阻塞/挂起链表); xEventListItem (事件链表项, 等待队列/信号量时使用); uxPriority (任务

优先级)；pxStack (栈起始地址)；pcTaskName (任务名称)；uxBasePriority (基础优先级，互斥量优先级继承时使用)。

三、调度机制

Q10. FreeRTOS 抢占式调度和时间片调度分别是什么？如何配置？

抢占式调度 (configUSE_PREEMPTION =

1)：高优先级任务就绪时，立即抢占当前运行的低优先级任务。

时间片调度 (configUSE_TIME_SLICING = 1，默认启用)：相同优先级任务轮流执行，每个 Tick 切换一次。

两者可同时启用：高优先级抢占，同优先级轮转。实际项目默认两者都开启。

Q11. PendSV 中断在 FreeRTOS 任务切换中扮演什么角色？

PendSV 是 ARM Cortex-M 专为 RTOS 上下文切换设计的异常，FreeRTOS 将其优先级设为最低。

任务切换流程：SysTick 触发 → 判断是否需要切换 → 若需要则 Pend PendSV → 所有更高优先级中断处理完毕后 PendSV 执行 → 保存当前任务寄存器到栈 → 切换 SP → 恢复新任务寄存器。

设为最低优先级保证任务切换发生在所有中断处理完成之后，不中断关键中断服务。

★ PendSV + SysTick 是任务切换核心机制，源码分析必考

Q12. vTaskStartScheduler() 调用后发生了什么？

1. 创建空闲任务 (prvIdleTask)，优先级为 0。
2. 若配置了软件定时器，创建定时器服务任务。
3. 调用 xPortStartScheduler() 配置 SysTick 和 PendSV 中断优先级。
4. 调用 prvStartFirstTask() (汇编)，触发 SVC 异常，恢复最高优先级就绪任务的上下文。
5. 正常情况下不会返回；若返回，说明堆空间不足（无法创建空闲任务）。

Q13. FreeRTOS 如何确定下一个要运行的任务？

使用宏 taskSELECT_HIGHEST_PRIORITY_TASK()，有两种实现方式：

通用方式 (configUSE_PORT_OPTIMISED_TASK_SELECTION =

0)：遍历就绪链表找最高优先级，O(n)。

优化方式 (= 1)：使用 ARM 的 CLZ (Count Leading Zeros) 指令，通过 uxTopReadyPriority 位图一条指令内找到最高就绪优先级，O(1)。STM32 Cortex-M4 支持硬件 CLZ，推荐开启。

四、中断管理

Q14. configMAX_SYSCALL_INTERRUPT_PRIORITY 的作用是什么？

定义了可以安全调用 FreeRTOS API 的最高中断优先级（数值越大优先级越低）。

优先级数值小于此宏（即优先级更高）的中断：不能调用任何 FreeRTOS API，不受调度器管理，保证最高实时性。

优先级数值大于等于此宏的中断：可调用带 FromISR 后缀的 FreeRTOS API。

STM32F407 典型配置：configMAX_SYSCALL_INTERRUPT_PRIORITY = 0x50（优先级5）。

★ STM32 NVIC 高4位有效，数值越小优先级越高，与 FreeRTOS 规则一致

Q15. 中断服务函数中为什么要用带 FromISR 后缀的 API?

普通 API 内部可能阻塞、触发任务切换，在中断中调用不安全。FromISR API 不阻塞，通过 pxHigherPriorityTaskWoken 参数通知是否需要切换任务：

```
BaseType_t xHigherPriorityTaskWoken = pdFALSE;
xQueueSendFromISR(xQueue, &data, &xHigherPriorityTaskWoken);
portYIELD_FROM_ISR(xHigherPriorityTaskWoken); // 决定是否触发任务切换
```

Q16. FreeRTOS 临界区如何进入和退出?

方式一（任务中使用）：

```
taskENTER_CRITICAL(); // 屏蔽低优先级中断，支持嵌套
/* 临界区 */
taskEXIT_CRITICAL(); // 恢复，嵌套计数归零才真正开中断
```

方式二（中断中使用）：

```
UBaseType_t uxSaved = taskENTER_CRITICAL_FROM_ISR();
/* 临界区 */
taskEXIT_CRITICAL_FROM_ISR(uxSaved);
```

注意：临界区内不能调用任何可能阻塞的 FreeRTOS API。

五、时间管理

Q17. vTaskDelay() 和 vTaskDelayUntil() 的区别?

vTaskDelay(xTicksToDelay)：从调用时刻起延时指定 tick，实际周期 = 任务执行时间 + 延时时间，有抖动。

vTaskDelayUntil(&xLastWakeTime,

xPeriod)：从上次唤醒时刻计算，保证固定周期调用，任务执行时间不影响周期，适合等时距采样：

```
TickType_t xLastWakeTime = xTaskGetTickCount();
for (;;) {
    vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(10)); // 精确10ms周期
    /* 传感器采样 */
}
```

★ 周期精度要求高的传感器采样任务必须用 vTaskDelayUntil

六、队列 (Queue)

Q18. FreeRTOS 队列的工作原理？线程安全吗？

FreeRTOS 队列是 FIFO 字节流缓冲区，采用数据拷贝（而非指针传递），内部通过临界区保证线程安全。写满时 Send 可选阻塞等待；读空时 Receive 可选阻塞等待。ISR 中使用带 FromISR 后缀的版本，不能阻塞。

```
// 创建：10个元素，每个元素 sizeof(MyData_t) 字节
QueueHandle_t xQ = xQueueCreate(10, sizeof(MyData_t));

// 发送（等待最多100ms）
xQueueSend(xQ, &data, pdMS_TO_TICKS(100));

// 接收（无限等待）
xQueueReceive(xQ, &buf, portMAX_DELAY);
```

Q19. xQueueSend、xQueueSendToBack、xQueueSendToFront 的区别？

xQueueSend == xQueueSendToBack：追加到队列尾部（默认 FIFO 行为）。

xQueueSendToFront：插入到队列头部（高优先级消息，实现 LIFO）。

xQueueOverwrite：队列满时强制覆盖最旧数据（仅适用于长度为 1 的队列）。

七、信号量 (Semaphore)

Q20. 二值信号量、计数型信号量、互斥量的区别？

二值信号量：值只有 0 和 1，用于任务同步（中断通知任务），不具有优先级继承。

计数型信号量：值范围 0~N，用于资源计数（如停车场空位数）或累计事件次数。

互斥量：用于互斥访问共享资源，具有优先级继承机制，防止优先级翻转。必须由同一任务获取和释放（不能在 ISR 中使用）。

★ 最重要区别：互斥量有优先级继承，二值信号量没有

Q21. 什么是优先级翻转？FreeRTOS 如何解决？

优先级翻转：低优先级任务 L 持有互斥量，高优先级任务 H 等待该互斥量，此时中优先级任务 M 抢占 L 运行，导致 H 实际上被 M 阻塞，H 的优先级被翻转到 M 以下。

FreeRTOS 解决方案——优先级继承：当高优先级任务 H 等待 L 持有的互斥量时，FreeRTOS 自动将 L 的优先级临时提升到 H 的级别，使 L 尽快释放互斥量。L 释放后恢复原优先级。

注意：优先级继承只能缓解，不能完全解决优先级翻转，复杂情况需在设计层面规避。

★ 优先级翻转与继承是 RTOS 面试经典必考题

Q22. 递归互斥量 (Recursive Mutex) 是什么? 什么时候使用?

允许同一任务多次获取同一互斥量而不死锁。每次 Take 计数+1, 每次 Give 计数-1, 计数归零时才真正释放。

```
// 配置: configUSE_RECURSIVE_MUTEXES = 1
SemaphoreHandle_t xMutex = xSemaphoreCreateRecursiveMutex();

// 同一任务可多次获取, 不死锁
xSemaphoreTakeRecursive(xMutex, portMAX_DELAY);
xSemaphoreTakeRecursive(xMutex, portMAX_DELAY);
xSemaphoreGiveRecursive(xMutex); // 两次 Give 才真正释放
xSemaphoreGiveRecursive(xMutex);
```

使用场景: 递归函数或嵌套调用链中需要获取同一互斥量保护共享资源时。

八、软件定时器

Q23. FreeRTOS 软件定时器的工作原理?

软件定时器由定时器服务任务 (Timer Daemon Task) 管理, 优先级由 configTIMER_TASK_PRIORITY 配置。

工作流程: 用户调用 xTimerStart 等 API → 向定时器命令队列发送命令 → 定时器服务任务从队列取命令处理 → 到期时在服务任务上下文中调用回调函数。

单次定时器 (pdFALSE): 到期后自动停止。周期定时器 (pdTRUE): 到期后自动重新计时。

★ 回调函数在定时器服务任务中运行, 绝对不能阻塞!

Q24. 软件定时器回调函数中能调用 vTaskDelay() 吗?

不能。回调函数在定时器服务任务上下文中执行, 若在其中调用 vTaskDelay(), 会阻塞定时器服务任务, 导致所有定时器的回调和命令处理都被延迟。

回调函数设计原则: 保持简短快速; 不调用阻塞型 API; 若需要耗时操作, 在回调中向队列发消息, 由专门任务处理。

九、内存管理

Q25. FreeRTOS 的 heap_1~heap_5 各有什么特点? 如何选择?

heap_1: 只能申请不能释放, 分配后永不回收。适合系统初始化时静态分配所有资源。

heap_2: 可申请和释放, 但不合并相邻空闲块 (会产生碎片)。适合每次申请相同大小的场景。

heap_3: 封装 C 标准库 malloc/free, 线程安全。堆大小由链接脚本决定。

heap_4: 最常用。可申请/释放, 合并相邻空闲块 (减少碎片)。内存池为连续数组。STM32F407 首选。

heap_5: 在 heap_4 基础上支持多块不连续内存区域 (多段堆)。适合 RAM 分散在多个地址空间的芯片 (如 STM32H7 内部+外部 SRAM)。

★ STM32F407 项目通常选 heap_4, STM32H7 外扩 SDRAM 选 heap_5

Q26. 如何确定 FreeRTOS 任务栈大小是否足够?

方法一：开发阶段开启栈溢出检测。

```
// FreeRTOSConfig.h
configCHECK_FOR_STACK_OVERFLOW = 2 // 启用溢出检测

// 实现钩子（溢出时回调）
void vApplicationStackOverflowHook(TaskHandle_t xTask, char *pcTaskName) {
    for(;;) // 死机便于调试
}
```

方法二：高水位线（High Water Mark）。

```
// 返回历史最小剩余栈空间（单位：字）
UBaseType_t uxRemaining = uxTaskGetStackHighWaterMark(taskHandle);

// 返回 0 或极小值说明栈接近溢出，需要增大
```

实践：先设较大值（512字）运行测试，观察 HighWaterMark，再缩小留 20% 余量。

十、移植与综合

Q27. 将 FreeRTOS 移植到 STM32 需要修改哪些关键文件?

1. 添加 FreeRTOS 源文件：tasks.c / queue.c / list.c / timers.c / event_groups.c / portable/GCC/ARM_CM4F/port.c / portable/MemMang/heap_4.c。
2. 修改 SYSTEM 文件（delay.c / sys.h）：delay 函数改为 RTOS 兼容版本。
3. 修改中断文件（stm32f4xx_it.c）：SysTick_Handler 调用 xPortSysTickHandler(); PendSV_Handler / SVC_Handler 由 FreeRTOS port.c 提供，不能重复定义。
4. 配置 FreeRTOSConfig.h：时钟频率、NVIC 优先级分组（必须选分组4）等。

□ 移植后若系统异常进入死循环，首先检查 NVIC 优先级分组是否设为 4 位全抢占（分组4）

Part 2 STM32 面试题 (17题)

一、架构基础

Q01. STM32F407 基于什么内核？有哪些主要特性？

基于 ARM Cortex-M4 内核，主频最高 168MHz，内置单精度 FPU 和 DSP 指令集。

主要特性：1MB Flash，192KB SRAM（含 64KB CCM）；3 个 12 位 ADC（最高 2.4MSPS）；2 个 DAC；14 个定时器；3 个 SPI / 3 个 I2C / 4 个 USART + 2 个 UART；USB OTG / 以太网 MAC / CAN / SDIO / FSMC；168 个 GPIO。

★ Cortex-M4 与 M3 最大区别：M4 有 FPU 和 DSP 指令

Q02. STM32 的存储器映射：主要区域的地址范围？

ARM Cortex-M 统一 4GB 地址空间，STM32F407 主要区域：

0x0000_0000 启动区域（映射 Flash 或 SRAM，取决于 BOOT 引脚）
0x0800_0000 内部 Flash（1MB）
0x1000_0000 CCM SRAM（64KB，仅 CPU 可访问，不支持 DMA）
0x2000_0000 内部 SRAM1（112KB）+ SRAM2（16KB）
0x4000_0000 APB1 外设基地址
0x4001_0000 APB2 外设
0x4002_0000 AHB1 外设（GPIO/DMA/RCC 等）
0xE000_0000 Cortex-M4 内核外设（NVIC/SysTick/ITM 等）

Q03. STM32 的启动方式有哪几种？BOOT0/BOOT1 如何配置？

BOOT0=0, 任意从内部 Flash 启动（正常运行模式，最常用）
BOOT0=1, BOOT1=0 从系统存储器启动（内置 BootLoader，用于 ISP 下载）
BOOT0=1, BOOT1=1 从内部 SRAM 启动（调试用）

启动过程：复位后 CPU 从 0x0000_0000 读取初始 SP，从 0x0000_0004 读取复位向量（PC），跳转到 Reset_Handler → SystemInit() → __main() → main()。

二、时钟系统

Q04. STM32F407 如何达到 168MHz 主频？HSE / HSI / PLL 各是什么？

HSE：外部高速时钟，探索者板载 8MHz 晶振；HSI：内部高速 RC，16MHz，精度较低；PLL：锁相环，对时钟倍频。

168MHz 配置路径：

```
HSE(8MHz) --[PLL_M=8]--> 1MHz
--[PLL_N=336]--> 336MHz
--[PLL_P=2]---> SYSCLK = 168MHz
```

```
AHB = 168MHz (不分频)
APB2 = 84MHz (AHB / 2)
APB1 = 42MHz (AHB / 4)
```

★ AHB 最大 168MHz, APB2 最大 84MHz, APB1 最大 42MHz, 面试常考

三、GPIO

Q05. STM32 GPIO 有哪几种工作模式?

输入模式: 浮空输入、上拉输入、下拉输入、模拟输入 (用于 ADC/DAC)。

输出模式: 推挽输出 (可主动驱动高低电平); 开漏输出 (只能拉低, 需外部上拉才能输出高)。

复用模式: 复用推挽 (UART TX / SPI MOSI 等); 复用开漏 (I2C SDA/SCL 等)。

★ I2C 用开漏+外部上拉, 因为多主机共享总线需要线与逻辑

Q06. 如何配置 GPIO 为复用功能? 以 UART1 TX (PA9) 为例。

```
_HAL_RCC_GPIOA_CLK_ENABLE();

GPIO_InitTypeDef GPIO_InitStructure = {0};
GPIO_InitStructure.Pin = GPIO_PIN_9;
GPIO_InitStructure.Mode = GPIO_MODE_AF_PP; // 复用推挽
GPIO_InitStructure.Pull = GPIO_NOPULL;
GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_HIGH;
GPIO_InitStructure.Alternate = GPIO_AF7_USART1; // 复用功能编号
HAL_GPIO_Init(GPIOA, &GPIO_InitStructure);
```

四、中断系统 (NVIC)

Q07. STM32 NVIC 优先级分组是什么? FreeRTOS 项目应选哪个分组?

STM32 使用高 4 位表示优先级 (16 个级别), 通过 PRIGROUP 将 4 位分为抢占和响应优先级:

```
分组0: 0位抢占 / 4位响应 16响应无抢占
分组1: 1位抢占 / 3位响应 2抢占 x 8响应
分组2: 2位抢占 / 2位响应 4抢占 x 4响应
分组3: 3位抢占 / 1位响应 8抢占 x 2响应
分组4: 4位抢占 / 0位响应 16抢占优先级 (FreeRTOS 推荐)
```

FreeRTOS 必须选分组4，这样 configMAX_SYSCALL_INTERRUPT_PRIORITY 的数值与抢占优先级直接对应，无歧义。

```
HAL_NVIC_SetPriorityGrouping(NVIC_PRIORITYGROUP_4);
```

□ FreeRTOS 项目若未使用分组4，configASSERT 会触发，程序进入死循环

五、定时器

Q08. STM32F407 三类定时器（基本/通用/高级）的主要区别？

基本定时器（TIM6/TIM7）：仅有计数/溢出功能，只能产生更新中断或触发 DAC，无 PWM 输出。

通用定时器（TIM2~TIM5，TIM9~TIM14）：16 位（TIM2/TIM5 为 32 位），支持 PWM 输出、输入捕获、输出比较、编码器接口，最常用（4 路 PWM 通道）。

高级定时器（TIM1/TIM8）：在通用定时器基础上增加互补输出（带死区控制）、刹车输入、重复计数器，专为电机控制（BLDC/FOC）设计。

Q09. 如何用定时器产生 1kHz、50% 占空比的 PWM？（TIM3 CH1，PA6）

定时器时钟 = APB1 x 2 = 84MHz。目标 1kHz = 周期 1ms：PSC=84-1，ARR=1000-1。

```
TIM_HandleTypeDef htim3 = {0};
htim3.Instance = TIM3;
htim3.Init.Prescaler = 84 - 1; // 计数频率 = 1MHz
htim3.Init.CounterMode = TIM_COUNTERMODE_UP;
htim3.Init.Period = 1000 - 1; // ARR, 周期 = 1ms
HAL_TIM_PWM_Init(&htim3);

TIM_OC_InitTypeDef sConfig = {0};
sConfig.OCMode = TIM_OCMODE_PWM1;
sConfig.Pulse = 500; // CCR, 占空比 = 500/1000 = 50%
sConfig.OCpolarity = TIM_OCPOLARITY_HIGH;
HAL_TIM_PWM_ConfigChannel(&htim3, &sConfig, TIM_CHANNEL_1);
HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);
```

六、串口 (USART/UART)

Q10. 串口空闲中断 (IDLE) 如何实现不定长数据接收？

空闲中断：RX 线上一帧数据传输完毕后出现超过 1 字节时间的空闲状态时触发。

不定长接收方案（DMA + IDLE 中断）：

```
// 1. 配置 USART DMA 接收（循环模式），数据写入固定缓冲区
HAL_UART_Receive_DMA(&huart1, rxBuf, sizeof(rxBuf));
```

```
// 2. 使能空闲中断
__HAL_UART_ENABLE_IT(&huart1, UART_IT_IDLE);

// 3. 在中断回调中处理
void USART1_IRQHandler(void) {
    if (__HAL_UART_GET_FLAG(&huart1, UART_FLAG_IDLE)) {
        __HAL_UART_CLEAR_IDLEFLAG(&huart1);
        HAL_UART_DMAStop(&huart1);
        uint32_t len = sizeof(rxBuf) - __HAL_DMA_GET_COUNTER(huart1.hdmarx);
        // 处理 rxBuf[0..len-1]
        HAL_UART_Receive_DMA(&huart1, rxBuf, sizeof(rxBuf)); // 重启
    }
}
```

这种方案 CPU 占用极低，是串口接收变长帧的工程标准做法。

★ IDLE + DMA 是串口不定长接收的最佳实践，面试必会

七、DMA

Q11. STM32F407 有几个 DMA 控制器？三种传输模式的区别？

两个 DMA 控制器，各 8 个流 (Stream)，每个流 8 个通道：

DMA1：连接 APB1 外设 (TIM2~7, USART2/3, I2C, SPI2/3 等)。

DMA2：连接 APB2/AHB 外设 (ADC, SPI1, USART1/6, SDIO, TIM1/8 等)。

三种模式：

正常模式 (Normal)：传输完成后停止，产生传输完成中断。

循环模式 (Circular)：传输完成后自动重新开始，适合持续采样 (ADC/串口)。

双缓冲模式 (Double Buffer)：两个内存缓冲区交替接收，一个缓冲被 DMA 填充时 CPU 处理另一个，实现零拷贝乒乓操作，适合高速数据流。

八、SPI / I2C

Q12. SPI 的四种工作模式 (CPOL/CPHA) 是什么？

CPOL (时钟极性)：空闲时时钟电平 (0=低, 1=高)。

CPHA (时钟相位)：数据在哪个沿采样 (0=第一个边沿, 1=第二个边沿)。

模式0: CPOL=0, CPHA=0 空闲低，上升沿采样 (最常用)

模式1: CPOL=0, CPHA=1 空闲低，下降沿采样

模式2: CPOL=1, CPHA=0 空闲高，下降沿采样

模式3: CPOL=1, CPHA=1 空闲高，上升沿采样

使用前查阅从器件手册确认支持的 SPI 模式。

九、看门狗与低功耗

Q13. IWDG 和 WWDG 的区别？

IWDG（独立看门狗）：时钟源为内部

LSI（约32kHz），独立于主时钟；任意时刻喂狗均可；适合防系统死机。

WWDG（窗口看门狗）：时钟源为

APB1，与系统同步；只能在计数值处于窗口内喂狗，过早或过晚均复位；适合检测程序时序是否正确。

★ IWDG 防死机，WWDG 防时序错误，实际项目通常只用 IWDG

Q14. STM32 三种低功耗模式的进入/退出方式？

Sleep 模式：CPU 停止，外设继续运行。进入：WFI/WFE；退出：任意中断/事件。

Stop 模式：CPU + 大多数外设停止，GPIO 保持，SRAM

保留。进入：HAL_PWR_EnterSTOPMode(...); 退出：EXTI 中断，唤醒后需重新初始化时钟。

Standby 模式：最低功耗（约2μA），SRAM

不保留，唤醒相当于复位。进入：HAL_PWR_EnterSTANDBYMode(); 退出：WKUP 引脚上升沿 / RTC 闹钟 / NRST。

十、HAL 库与综合

Q15. volatile 关键字在嵌入式 C 中有什么作用？举两个场景。

volatile 告诉编译器：该变量随时可能在程序控制之外被改变，禁止编译器对该变量进行读取缓存优化（每次必须从内存读取）。

场景1——外设寄存器：硬件寄存器值随时可能改变（如 ADC 数据寄存器），必须声明为 volatile。STM32 HAL 库中寄存器均通过 volatile 结构体指针访问。

场景2——中断/多任务共享变量：

```
volatile uint8_t g_uart_rx_flag = 0; // ISR 写，主循环读  
// 若不加 volatile，编译器可能缓存读取结果，主循环永远看不到更新
```

□ volatile 不能替代互斥锁，不保证操作的原子性

Q16. 什么是大端模式和小端模式？STM32 使用哪种？

大端（Big-Endian）：高字节存放在低地址。如 0x12345678 存为 12 34 56 78。

小端（Little-Endian）：低字节存放在低地址。如 0x12345678 存为 78 56 34 12。

STM32（ARM Cortex-M）默认使用小端模式。

工程影响：从串口/网络接收大端字节流时需手动做字节序转换：

```
// 大端字节流解析（如 FreeRTOS 冷链项目中温度帧解析）
```

```
short temp_raw = (short)((frame[2] << 8) | frame[3]);  
double temp = temp_raw / 10.0;
```